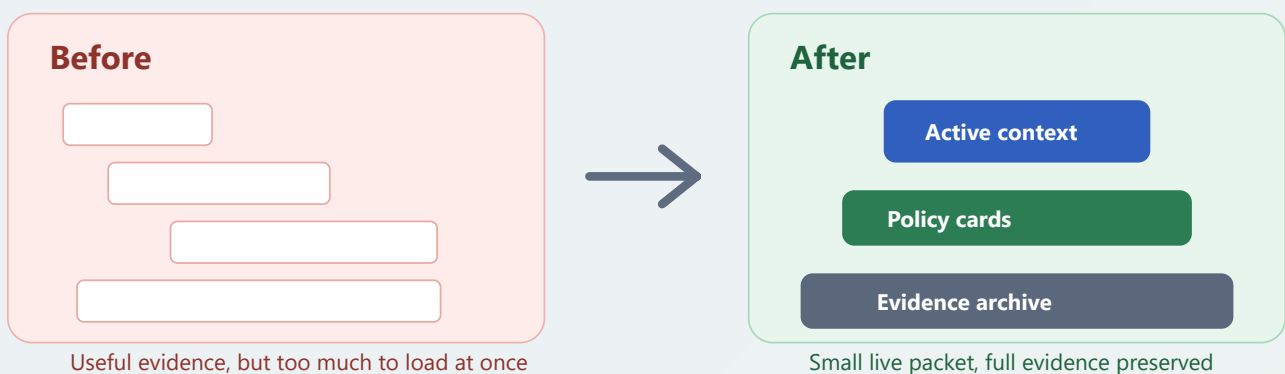


The new context system, explained simply

What was implemented, why it matters, and how it helps future Pokemon training sessions stay focused instead of drowning in notes.

Implemented May 14, 2026 in docs/pokemon_mastery



What changed?

The Pokemon mastery project had become very valuable, but also very large. It had replays, reviews, quick tests, score ledgers, strategy notes, local romhack mechanics notes, and source-to-policy notes. The risk was not that the notes were bad. The risk was that a future session could load too much, load the wrong things, or follow the latest lesson too rigidly.

I implemented a small control layer above the big archive. Think of it as a dashboard on top of a library. The library is still there. The dashboard tells the next session which shelf to open, which rule matters right now, and which score proves progress.

1. ACTIVE CONTEXT

A short startup packet that says what matters now and what should not be loaded by default.

2. POLICY CARDS

Small decision cards for the current recurring mistakes, each with a default rule and the opposite boundary.

3. REAL SCORING GATES

Progress is measured by fresh replay decisions, not by how many notes were written.

In simple terms: the new system tells future me, "Start here, load only what you need, make a decision, score it, and keep the archive as proof."

Files created or updated

File	Plain-English job
<code>active_context.md</code>	The small packet future sessions should read first.
<code>context_management_plan.md</code>	The approved plan, comparisons, vote record, and caveats.
<code>policy_cards/</code>	Compact rules for the active decision boundaries.
<code>mechanics_pending_index.md</code>	A warning board for romhack mechanics that need local proof.
<code>master_index.md</code>	Updated so future work routes through the new compact layer.

WHY THIS WAS NEEDED

The old problem was not "bad notes." It was too much context.

The project already had a lot of strong material. The trouble was that strong material can still become hard to use. If every session starts by reading giant files, the important lesson can get buried. If every replay miss creates another long note, the project can feel smarter while the move choices do not actually improve.

BEFORE: THE FAILURE PATTERN

- Many useful docs existed, but the next best file was not always obvious.
- Large strategy ledgers mixed core rules with replay history.
- Fresh replay misses could create more notes without proving transfer.
- The newest lesson could overcorrect the next decision.
- Vanilla GSC knowledge could leak into romhack advice too confidently.

AFTER: THE NEW BEHAVIOR

- Start with one compact context packet.
- Load one to three policy cards instead of whole archives.
- Use fresh replay packets as the main scoreboard.
- Require the opposite boundary so rules do not become scripts.
- Check local mechanics status before romhack claims.

The goal is not fewer documents forever. The goal is that each future work session has a small working memory and a clear scoreboard.

A plain example

Situation: a sleeping Pokemon is on the field

Old risk: I might remember a recent lesson like "preserve sleepers for Sleep Clause" and switch automatically.

New rule: the policy card says this is common, but not absolute. I must ask: does the sleeper have an active job right now, or is it more valuable saved for later?

Switch

Stay

Autopilot

The evidence pyramid

The core design is a three-layer pyramid. The top layer is tiny and used every time. The middle layer holds compact rules. The bottom layer is the full evidence archive.

active_context.md: what to load now

policy_cards: current decision boundaries

archive: replays, reviews, ledgers, fixtures, source notes

TOP LAYER

The short packet. It lists the current objective, the active mistakes, the score gates, and the correct startup files.

MIDDLE LAYER

Policy cards. Each card says when a rule applies, when it flips, how it fails, and what evidence supports it.

BOTTOM LAYER

The full archive stays intact. It is used for audits, evidence, and deeper study, not loaded blindly every time.

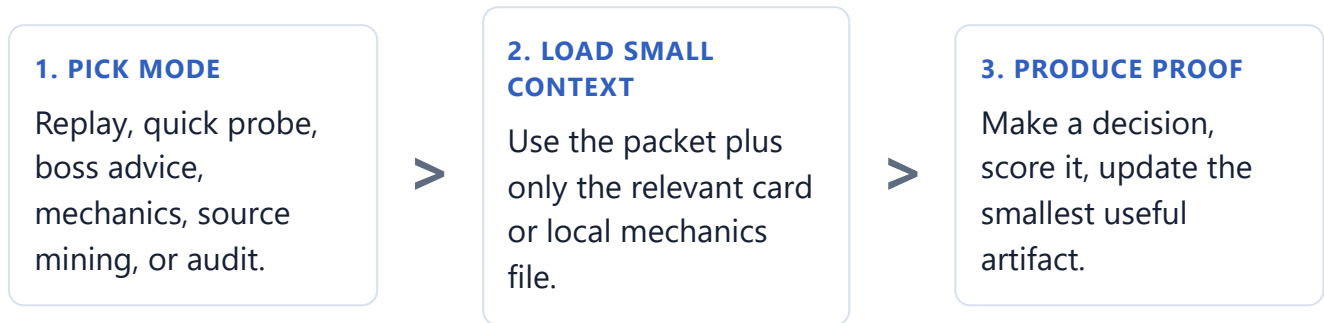
Why this is better

It solves the context problem without throwing away the work already done. Future sessions can move faster because the current high-value knowledge is extracted into small cards, while the original replay and review evidence still exists if a rule needs to be checked.

The big archive is now treated like a library. The active context packet is the reading list. Policy cards are bookmarks for the current lessons.

The startup packet keeps each session on rails

The active context packet is a compact briefing for future work. It says: here is the current goal, here are the active mistakes, here is how to use web search, here is when to use subagents, and here is what counts as progress.



The session header

The packet adds a small form to fill out before serious work. It prevents vague sessions like "study more Pokemon" and forces the work to name the current job.

```
Goal:
Mode: replay / quick_probe / boss_advice / mechanics / source_mining / audit
Information model:
Current bottleneck:
Artifact target:
Forbidden info:
Stop condition:
```

WITHOUT THE HEADER	WITH THE HEADER
"I should read more GSC strategy." This can become broad note-taking.	"Replay mode. Bottleneck: sleep absorber assignment. Forbidden info: future turns. Stop after 30 scored decisions." This creates a measurable rep.

Policy cards turn lessons into usable decision boundaries

A policy card is a small rule sheet. It is not a long essay. It tells me when a rule applies, what the default is, when the opposite play is right, how the rule can fail, and where the evidence lives.

CREATED CARDS

- Cash-out boundary
- Sleep absorber and set ambiguity
- Hazard loop and Rapid Spin window
- Support handoff after a job lands
- Romhack mechanics firewall

THE IMPORTANT DESIGN CHOICE

Each card must include an opposite boundary. That keeps a lesson from becoming a rigid script.

Example: cash-out boundary

Should a weak support Pokemon be preserved or spent?

Default: preserve it if it still has a route-defining job, like keeping hazards, checking a threat, or enabling a future switch.

Opposite boundary: spend it now if Explosion or a sacrifice opens a concrete converter that slow play cannot reach.

Worst failure: saving it just because it is famous or valuable, then later realizing it had no safe entry and no real job.

Trigger

Default

Opposite boundary

Exceptions

Worst branch

Evidence

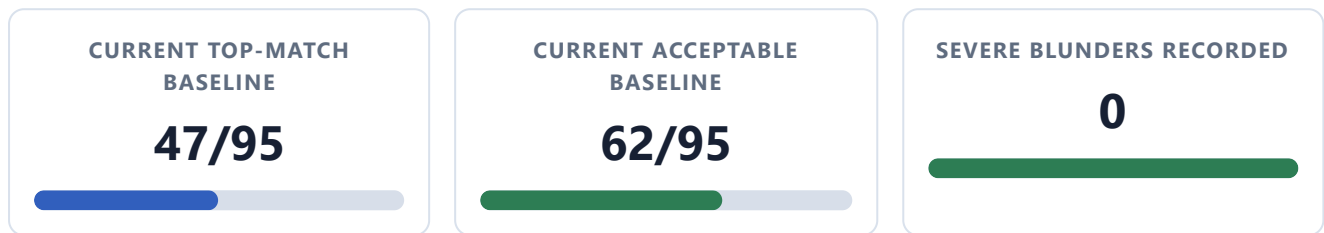
Drill

The card is a reminder to ask the right question, not an answer to every position. It points to the archive if the rule needs to be checked.

HOW PROGRESS IS MEASURED

The scoreboard changed from "more notes" to "better unseen decisions"

The biggest measurement change is philosophical but important: writing more artifacts does not prove improvement. Improvement has to show up in fresh positions that were not already turned into a lesson.



Primary gate: sealed replay-transfer packet

A sealed packet means fresh replay turns where the future outcome and answer labels are hidden before the move choice is made. This is the main proof because it tests transfer to unseen positions.

Gate	Plain meaning
30-50 decisions	Enough turns to reduce one-lucky-run noise.
55% top-match target	The chosen move matches the expert replay move often enough to show direction.
70% acceptable target	Even when not exact, the move is still strategically acceptable.
0 severe blunders	No move that clearly breaks the live win route.
No repeated uncorrected error twice	The same mistake should not keep appearing inside one packet.

These numbers are provisional. The plan says they should be recalibrated after one or two sealed packets.

Small lab tests are useful, but they are not the final exam

The project has many constructed quick probes. These are useful because they check whether a specific lesson was understood. But they are often written after the mistake is already known, so they can be too easy to pass. That is why the plan separates them from fresh replay evidence.

GOOD USE OF A PROBE

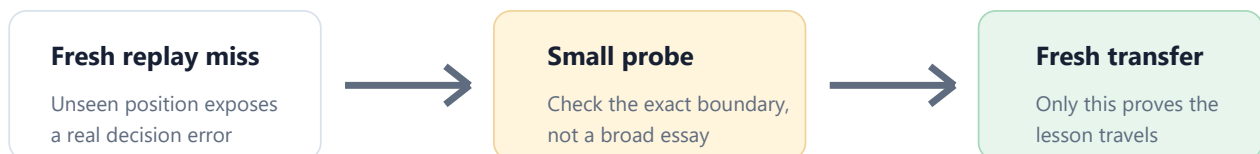
A replay showed I over-preserved a low Cloyster. A tiny probe tests the boundary: preserve when it still has a job, cash out when the converter is ready.

BAD USE OF A PROBE

Writing many labeled scenarios until the lesson becomes recognizable by name, then treating a perfect score as proof of mastery.

The new rule

By default, create at most one small constructed probe for each fresh replay miss. If one replay exposes several truly different mistakes, the plan allows separate probes for separate boundaries. "Retire or merge" means mark or index the probe after transfer, not delete it.



The mechanics firewall stops confident wrong advice

This project is not just vanilla GSC. It is a Pokemon Gold romhack. That means outside GSC knowledge is useful, but it is not automatically true locally. The mechanics pending index makes that visible.

Simple rule: if a move recommendation depends on local mechanics, do not settle it with memory or web search. Check local docs, source, fixtures, debugger output, or emulator traces.

EXAMPLES OF MECHANICS THAT NEED STATUS

- Spikes layer count and successful Rapid Spin hazard clearing.
- Sleep duration, wake timing, Sleep Talk, and Rest.
- Explosion timing, Ghost interactions, and damage behavior.
- Type passives, Focus Band, Quick Claw, Counter, Mirror Coat.
- What ordinary boss AI is allowed to know.

STATUS LABELS

- `runtime_verified`
- `source_verified`
- `supplied_unverified`
- `unknown`
- `not_decision_relevant`

Example

If the decision is "set Spikes because Rapid Spin works like vanilla," the firewall stops that. The local index now marks the romhack Spikes layer count and successful Rapid Spin hazard-clear command as `runtime_verified`, while exact turn timing and failure branches still need fixtures. The answer should use the verified facts but cap confidence on the unknown timing details.

WHY THIS MATTERS

A bad move caused by wrong mechanics is not the same as a bad move caused by bad strategy. The new index keeps those error classes separate.

Autonomy is now controlled, not vague

The plan does not say "never use subagents" or "always use all agents." It says use the amount of help that improves the work without turning each session into process management.

DEFAULT: SPECIALIST BURSTS

The main session stays responsible. A subagent gets one bounded job, such as source scouting, mechanics checking, replay proctoring, measurement scoring, or context stewardship.

RARE: STANDING COUNCIL

Use a full council only for declared audits or architecture work, like the one that approved this plan.

Every subagent packet must say

Objective What exact question is being answered?	Allowed sources What files or web sources may it use?	Forbidden info What must it not reveal or inspect?
Output schema What compact answer should it return?	Stop condition When should it stop instead of wandering?	Max references How much evidence is enough?

Web search rules

Use web search when	Do not use web search when
Choosing fresh Smogon/GSC material, checking current competitive sources, or extracting a new source-to-policy rule.	Answering a sealed replay turn before freezing the move.
Investigating a repeated error that local notes do not explain.	Settling local romhack mechanics that need local evidence.

Every implementation idea had to pass on its own

The user clarified that approval was needed for each individual idea, not just the plan as a whole. So the plan used five voters: the main assistant plus four subagents. An idea needed at least three yes votes.

24

ATOMIC IDEAS VOTED

Each one was listed separately before implementation.

24

MAJORITY APPROVALS

All implementation ideas passed the 3-of-5 threshold.

3

IMPORTANT CAVEATS

Baseline, score thresholds, and probe caps were approved with limits.

The caveats that matter

- Runs 052-057 are a current baseline, not proof of overall mastery.
- The 55% top-match and 70% acceptable-match targets are provisional.
- The probe cap is a default. If one replay exposes multiple distinct errors, more than one tiny probe can be justified.

This means the implementation was not just "my gut." It was compared, challenged, voted on, and written with the caveats preserved.

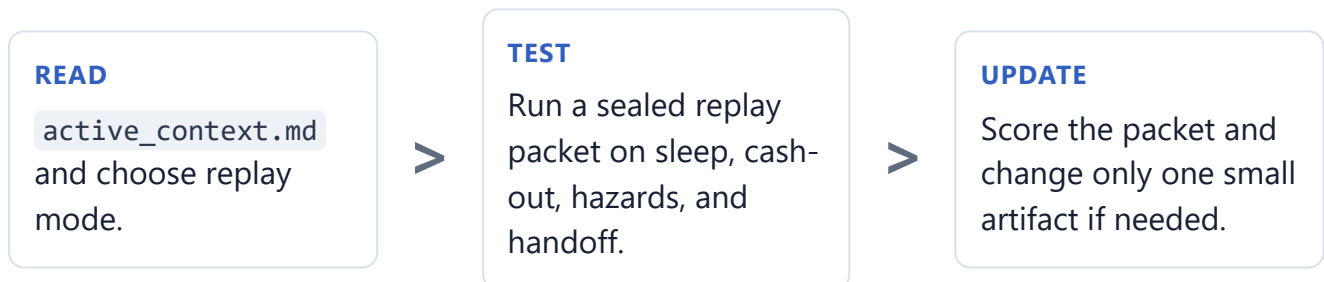
Why this is better

The vote process matters because context systems can easily become attractive busywork. The council pushed the implementation toward small, measurable, low-tooling changes instead of a full reorganization or a big retrieval app.

How a future training session should feel different

Moment	Before	After
Starting a session	Open many docs and hope the right lesson stands out.	Open <code>active_context.md</code> , choose mode, and load one relevant card.
Using a lesson	Risk turning the latest miss into a hard rule.	Check the default and the opposite boundary.
Measuring improvement	More notes can feel like progress.	Fresh replay-transfer packets are the scoreboard.
Handling romhack claims	Vanilla knowledge can sneak in as local truth.	Mechanics status must be known or confidence is capped.
Using subagents	Could become broad or uncontrolled.	Use bounded specialist bursts, councils only for audits.

Concrete next use



The system is better because it protects the real goal: stronger move choices in unseen positions. It keeps the notes useful, but it stops the notes from becoming the goal.

The implemented system in one picture

